

Universidad de San Andrés, *Buenos Aires, Argentina*



I206 - Inferencia y Estimación

Trabajo Práctico II

Teoría de la Información:

Codificación de Huffman

Camiscia, Luca

lcamiscia@udesa.edu.ar

Palavecino, Axel

apalavecino@udesa.edu.ar

Pereyra, Agustín

apereyra@udesa.edu.ar

23 de octubre de 2025

Abstract

En este presente trabajo se aborda experimentalmente este principio, investigando cómo se desempeña en la práctica el algoritmo de **Codificación de Huffman**, donde se investiga cómo la capacidad predictiva de una fuente determina su potencial compresión, esto se hace mediante el análisis de textos con propiedades de ocurrencia de caracteres radicalmente diferentes (lenguaje natural, datos estructurados y secuencias genómicas), además de demostrarse empíricamente la optimalidad de este método. Los resultados cuantifican la eficiencia del algoritmo, que minimiza la longitud de codificación promedio ... (**Aca van el resto de los resultados**)

1. Introducción

En la era del **Big Data**, obtener capacidad de representar información de manera concisa y compacta es un desafío fundamental. La *Teoría de la Información*, la cual comenzó **Claude Shannon**,

postula que la compresibilidad de cualquier dato está intrínsecamente limitada por su contenido de incertidumbre, esta medida es cuantificable, y es conocida como *entropía*.

2. Desarrollo Experimental

2.1. Estimación del Modelo probabilístico de la Fuente

El primer paso en la implementación de la **codificación de Huffman** es estimar el modelo probabilístico de la fuente (texto). Esto se logra mediante el análisis de la frecuencia de aparición de cada símbolo en el conjunto de datos. La probabilidad estimada p_i del i -ésimo símbolo s_i se calcula como:

$$p_i = \frac{f_i}{\sum_{j=1}^N f_j} \quad (1)$$

donde cada caracter es una realización de la variable aleatoria fuente S . El objetivo en esta etapa es estimar una **función de masa de probabilidad (PMF)** de cada fuente de texto. El resultado de este paso es un conjunto de pares (s_i, p_i) que representan los símbolos y sus probabilidades de ocurrencia, es decir, su **probabilidad empírica** de aparición $p(x)$. Este diccionario constituye el modelo estadístico de la fuente, y es el insumo fundamental sobre el cual operará el algoritmo de **Huffman**.

Es de interés aclarar que en este informe no se realiza un distinción entre mayúsculas y minúsculas, ya que el objetivo principal es reducir el tamaño del texto manteniendo su compresibilidad. La información que aporta una diferenciación entre mayúscula y minúscula es muy pequeña al lado de la complejidad que le aporta a la codificación de Hoffman. La ausencia de mayúsculas al inicio de las oraciones no afecta la comprensión, por lo que no se considera prescindible. No obstante, posteriormente podría aplicarse un algoritmo que reemplace automáticamente por mayúsculas las letras que sigan a un punto (“.”).

Las características del resultado de la Ecuación (1) se basan en los **Axiomas de la Probabilidad**,

es decir, la suma de los p_i es igual a 1, $\sum_{i=1}^N p_i = 1$, la probabilidad de un evento puntual es mayor a cero, $\mathbb{P}(p_i) \geq 0$, y si $A \cap B = \emptyset \Rightarrow \mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B)$

2.2. Codificación de Huffman

La característica que diferencia este código de otros universales como el *código Morse* es que ninguna cadena de código asignada a algún símbolo es prefijo de otra. A esta propiedad se la conoce como *prefix-free* y es gracias a esto que su decodificación es inmediata.

El código se construye como un árbol binario, cuyas hojas representan un símbolo y a cada rama un elemento binario. Los caracteres de menor frecuencia se encuentran en los mayores niveles de profundidad del árbol, lo que tiene sentido porque los más frecuentes serán los más rápidos de decodificar y esto es justamente lo que buscamos.

2.3. Construcción del Árbol de Huffman

Algorithm 1 HuffmanCoding

Input: pmf $p(s)$ de los caracteres s_i de la cadena S
 $Q \leftarrow$ priority-queue de nodos $n_i = \langle s_i, p(s_i) \rangle$
orden ascendente de p
while $\text{len}(Q) > 1$ **do**
 $n_1 \leftarrow Q.\text{pop}()$
 $n_2 \leftarrow Q.\text{pop}()$
 $n_{\text{padre}} \leftarrow \langle n_1 | n_2, n_1.p + n_2.p \rangle$
 $Q.\text{enqueue}(n_{\text{padre}})$
end while
 $\text{root} \leftarrow Q.\text{pop}()$
return root

Se añaden los símbolos de menor a mayor frecuencia en el árbol, creando subárboles de manera que la frecuencia del nodo padre es la suma de la frecuencia de sus hijos. De esta manera, iterativamente se van uniendo los subárboles dejando una estructura como la de la Figura ?? Luego es tan simple como asignar a cada rama un valor binario y la codificación para cada símbolo es el camino que hay que hacer desde la raíz hasta la hoja que representa a ese símbolo. En el ejemplo de la figura: a = 0, b = 10, c = 110 y d = 111

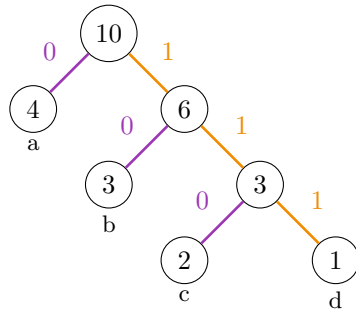


Figura 1: Árbol de Huffman para la cadena $S = abcdabcaba$

3. Análisis de Datos

ACA VAN LOS GRAFICOS DE BARRA

En los graficos se presentan las distribuciones de frecuencia de aparición de símbolos en tres archivos de texto distinto: adn.txt, mitologia.txt y tabla.txt. **SI ESTO LO PONEN EN METODOLOGIA AVISEN**

En primer lugar, el archivo adn.txt muestra una distribución completamente uniforme entre los cuatro símbolos que lo componen. Cada uno aparece con una frecuencia cercana al 25 % lo cual es coherente con la estructura de secuencia de un ADN.

Por otro lado, el archivo mitologia.txt muestra una distribución de frecuencias muchos mas variadas. Notamos como unas pocas letras concentran una gran cantidad de las apariciones. Esto podria deberse a la estructura del lenguaje español. Asimismo, vemos como el caracter mas usado es el (espacio) lo que concuerda con la regla gramatical de separar las palabras con este simbolo. **(Capaz puede decir algo mas)**

Finalmente, en el ultimo archivo al ser una tabla de valores observamos como predominan los caracteres numericos y signos de puntuacion. A diferencia del caso anterior, esta distribución no sigue parametros linguisticos sino de organizacion tipica de informacion numerica.

En conjunto, estas diferencias exhiben como las distribuciones de frecuencia estan fuertemente de-

2.4. Medidas de Desempeño y Análisis Cuantitativo

Longitud Promedio de Codificación (L):

$$L = \sum_{i=1}^N p_i l_i \quad (2)$$

Comparación con la Codificación Uniforme

$$L_{\text{unif}} = \lceil \log_2(N) \rceil \quad (3)$$

Reducción de Memoria

$$R = \left(1 - \frac{L}{L_{\text{unif}}} \right) \times 100 \% \quad (4)$$

Compresibilidad de la Fuente (Entropía Normalizada, η)

$$\eta = \frac{H(S)}{\log_2(N)} \quad (5)$$

terminados por la naturaleza de la informacion

AHORA VA LA TABLA

Los tres textos presentan comportamientos muy distintos, lo que permite observar como la estructura estadística mencionada anteriormente influye directamente en su compresibilidad

El archivo adn.txt representa el extremo mas simple. Su distribución uniforme produce una entropía baja de 1.99 bit/símbolo (1 normalizada) y una longitud promedio de Huffman igual a la mínima posible (2 bits/símbolo). Ante la inexistencia de diferencias en las probabilidades, el código no puede optimizarse, por lo que resulta en una reducción de memoria nula. Este es el peor caso posible donde la codificación de Huffman no ofrece ninguna ventaja.

Por otra parte, la distribución variada de mitologia.txt típica del lenguaje natural genera una entropía intermedia 4.26 bits/símbolo (0.79 normalizada), y permite al algoritmo asignar códigos mas cortos a los símbolos frecuentes. Gracias a esto, se obtiene una longitud promedio de 4.27 bits/símbolo y una reducción de memoria de al 28.31 %. En este caso se puede apreciar claramente como la redundancia del lenguaje natural puede aprovecharse para comprimir informacion de manera eficiente.

Por su parte, tabla.txt es aquel en el que se logra una mayor compresibilidad siendo de 41.30 %. Esto

debido a que su frecuencia esta dominada por unos pocos caracteres numericos y signos de puntuacion. Esta fuerte concentracion reduce la entropia a solo 3.46 bits/simbolo (0.64 normalizada) y permitiendo obtener la mayor eficiencia de codificacion.

4. Conclusión